

Задание №8

Вопрос №1

Условие:

Создайте смарт-контракт по образцу тех, которые рассматривались и создавались на практикуме.

Ответ:

Ниже представлен оригинальный смарт-контракт `CertificateIssuer`, разработанный на языке Solidity. Контракт предназначен для автоматизированной выдачи, учёта и отзыва цифровых сертификатов об обучении или квалификации.

Юридическая характеристика смарт-контракта

С правовой точки зрения данный смарт-контракт реализует следующие отношения:

- 1) Издатель сертификатов (владелец контракта) является уполномоченным лицом, которое подтверждает прохождение обучения или получение квалификации.
- 2) Получатель сертификата (обучающийся) приобретает право на получение цифрового документа, подтверждающего его компетенции.
- 3) Сертификат имеет срок действия, после истечения которого он автоматически теряет юридическую силу.
- 4) Издатель вправе отозвать сертификат досрочно в случае выявления нарушений.
- 5) Запись в маппинге `certificates` является юридически значимым подтверждением наличия сертификата у конкретного лица.

Код смарт-контракта

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 /**
5  * @title CertificateIssuer
6  * @dev
7  *
8  *
9  */
10 contract CertificateIssuer {
11
12     address public owner; //
13
14     uint256 public certificateCounter; //
15
16     //
17     struct Certificate {
18         uint256 id; //
```

```

18         address recipient;                                //
19         string fullName;                                  //
20         string courseName;                                //
21         uint256 issueDate;                                //
22         uint256 expiryDate;                                //
23         bool isActive;                                    //
24     }
25
26     //
27     mapping(address => Certificate) public certificates;
28
29     //
30     mapping(address => bool) public hasActiveCertificate;
31
32     //
33     event CertificateIssued(address indexed recipient, uint256 indexed
34         certificateId, string courseName);
35     event CertificateRevoked(address indexed recipient, uint256
36         indexed certificateId, string reason);
37     event CertificateExpired(address indexed recipient, uint256
38         indexed certificateId);
39
40     modifier onlyOwner() {
41         require(msg.sender == owner, "Only owner can perform this
42             action");
43         _;
44     }
45
46     modifier certificateExists(address _recipient) {
47         require(hasActiveCertificate[_recipient], "No active
48             certificate for this address");
49         _;
50     }
51
52     constructor() {
53         owner = msg.sender;
54         certificateCounter = 0;
55     }
56
57     /**
58     * @dev
59
60     * @param _recipient
61     (
62

```

```

55     * @param _fullName
56
57     * @param _courseName
58
59     * @param _validityDays
60
61     (0 =
62
63     ).
64
65     */
66
67     function issueCertificate(
68         address _recipient,
69         string memory _fullName,
70         string memory _courseName,
71         uint256 _validityDays
72     ) public onlyOwner {
73         require(_recipient != address(0), "Recipient address cannot be
74             zero");
75         require(!hasActiveCertificate[_recipient], "Recipient already
76             has an active certificate");
77
78         certificateCounter++;
79         uint256 issueTimestamp = block.timestamp;
80         uint256 expiryTimestamp = (_validityDays == 0) ? 0 :
81             issueTimestamp + (_validityDays * 1 days);
82
83         Certificate memory newCertificate = Certificate({
84             id: certificateCounter,
85             recipient: _recipient,
86             fullName: _fullName,
87             courseName: _courseName,
88             issueDate: issueTimestamp,
89             expiryDate: expiryTimestamp,
90             isActive: true
91         });
92
93         certificates[_recipient] = newCertificate;
94         hasActiveCertificate[_recipient] = true;
95
96         emit CertificateIssued(_recipient, certificateCounter,
97             _courseName);
98     }
99
100     /**
101     * @dev
102
103     * @param _recipient
104     * @param _reason
105
106     (
107
108     ).
109
110     */
111
112     function revokeCertificate(address _recipient, string memory
113         _reason) public onlyOwner certificateExists(_recipient) {
114         Certificate storage cert = certificates[_recipient];
115         require(cert.isActive, "Certificate is already revoked or
116             expired");
117     }

```

```

97         cert.isActive = false;
98         hasActiveCertificate[_recipient] = false;
99
100         emit CertificateRevoked(_recipient, cert.id, _reason);
101     }
102
103     /**
104     * @dev
105
106     * @param _recipient
107     * @return bool
108     */
109     function isCertificateValid(address _recipient) public view
110     certificateExists(_recipient) returns (bool) {
111         Certificate memory cert = certificates[_recipient];
112         if (!cert.isActive) {
113             return false;
114         }
115         if (cert.expiryDate != 0 && block.timestamp > cert.expiryDate)
116         {
117             return false;
118         }
119         return true;
120     }
121
122     /**
123     * @dev
124
125     * @param _recipient
126     * @return Certificate
127     */
128     function getCertificate(address _recipient) public view
129     certificateExists(_recipient) returns (Certificate memory) {
130         return certificates[_recipient];
131     }
132
133     /**
134     * @dev
135
136     *
137
138     * @param _recipient
139     */
140     function expireCertificate(address _recipient) public onlyOwner
141     certificateExists(_recipient) {
142         Certificate storage cert = certificates[_recipient];
143         if (cert.expiryDate != 0 && block.timestamp > cert.expiryDate)
144         {
145             cert.isActive = false;
146             hasActiveCertificate[_recipient] = false;
147             emit CertificateExpired(_recipient, cert.id);
148         }
149     }

```

```

139     }
140 }
141
142 /**
143  * @dev
144  *
145  * @param _newOwner
146  */
147 function transferOwnership(address _newOwner) public onlyOwner {
148     require(_newOwner != address(0), "New owner cannot be zero
149         address");
150     owner = _newOwner;
151 }

```

Пояснение к ключевым элементам смарт-контракта

1) **Структура Certificate.** В отличие от образца, здесь используется структура, которая объединяет все данные о сертификате: уникальный идентификатор, адрес получателя, полное имя, название курса, даты выдачи и истечения срока, а также статус активности.

2) **Счётчик certificateCounter.** Каждому новому сертификату присваивается уникальный номер, что позволяет вести учёт и ссылаться на конкретный документ.

3) **Механизм срока действия.** Добавлена возможность указывать срок действия сертификата в днях (параметр `_validityDays`). Если указано 0, сертификат считается бессрочным. Функция `isCertificateValid` проверяет не только статус, но и не истёк ли срок.

4) **Функция expireCertificate.** Позволяет владельцу контракта вручную обновлять статус сертификатов с истёкшим сроком для поддержания актуальности базы данных.

5) **Функция transferOwnership.** Позволяет передать права управления контрактом другому лицу (например, при смене уполномоченного сотрудника образовательной организации).

Отличия от рассмотренных на практикуме образцов

- 1) Вместо лицензий на контент — сертификаты об обучении.
- 2) Добавлена структура данных для хранения расширенной информации.
- 3) Введён механизм срока действия сертификатов.
- 4) Добавлена функция принудительного истечения срока.
- 5) Добавлена функция передачи прав собственности на контракт.
- 6) Изменены названия всех функций и переменных.

Юридические аспекты применения данного смарт-контракта

1) **Правовой статус цифрового сертификата.** Цифровой сертификат, выпущенный через смарт-контракт, может признаваться электронным документом при условии, что выпускавшая организация имеет соответствующий статус и сертификат подписан с использованием усиленной квалифицированной электронной подписи (в российском правовом поле). Данный смарт-контракт использует адрес владельца как аналог подписи.

2) **Договорная основа.** Смарт-контракт может быть интегрирован с внешним интерфейсом, где обучающийся соглашается с условиями получения сертификата и оплачивает услугу (в код можно добавить функцию оплаты).

3) **Отзыв сертификата.** Функция `revokeCertificate` позволяет издателю аннулировать сертификат при выявлении нарушений, что соответствует праву организации пересмотреть решение о выдаче документа.

4) **Доказательственная сила.** Записи в блокчейне о выдаче, отзыве и проверке сертификатов могут использоваться в качестве доказательств в судебных спорах о квалификации лица или факте прохождения обучения.

5) **Рекомендации по внедрению.** Перед использованием в реальной деятельности рекомендуется: - Провести аудит кода на предмет уязвимостей. - Определить порядок идентификации получателей (привязка адреса кошелька к реальному лицу). - Обеспечить хранение персональных данных в соответствии с Федеральным законом № 152-ФЗ.